

贪心算法

《算法设计与分析》

信息科学与技术学院 曾华琳



02



任务选择问题



二、任务选择问题

- 假定有一系列任务的集合 $S = \{a_1, a_2, \dots, a_n\}$ ，这些任务竞争同一资源，这个资源一次只能被一个任务占用。任务 a_i 占用该资源后，不能中断，直至完成。每个任务 a_i 有一个起始时间 s_i 和一个完成时间 f_i ，且 $0 \leq s_i < f_i < \infty$ 。如果任务被选择占用资源，则该任务可在半开时间区间 $[s_i, f_i)$ 内占用资源。如果任务 a_i 的活动区间 $[s_i, f_i)$ 和任务 a_j 的活动区间 $[s_j, f_j)$ 互不重叠，则称任务 a_i 和 a_j 是相互兼容的，即他们占用资源时，互不冲突。
- 任务选择(Task Select)问题就是在任务占用资源互不冲突的条件下，选择尽可能多的任务占用资源，即要选择相互兼容任务的最大子集。



贪心途径

- 选择一个最大可相容子集:
 - 选择 $a_m \in S_{ij}$ a_m 是最先完成的(贪心选择)
 - 将 a_m 加入最优解的集合
 - 在 S_{mj} 中解决同样的问题
 - 我们通过选择 a_m 已经保证了要在最优解中用到这个任务
所以在做选择之前我们并不需要解决子问题 S_{mj}
 - 这个问题具有贪心选择性质



描述子问题

- 原问题：在 s 中寻找最大可相容子集
- 任务已经按照其结束顺序升序排序

$$a_0, a_1, a_2, a_3, \dots, a_{n+1}$$

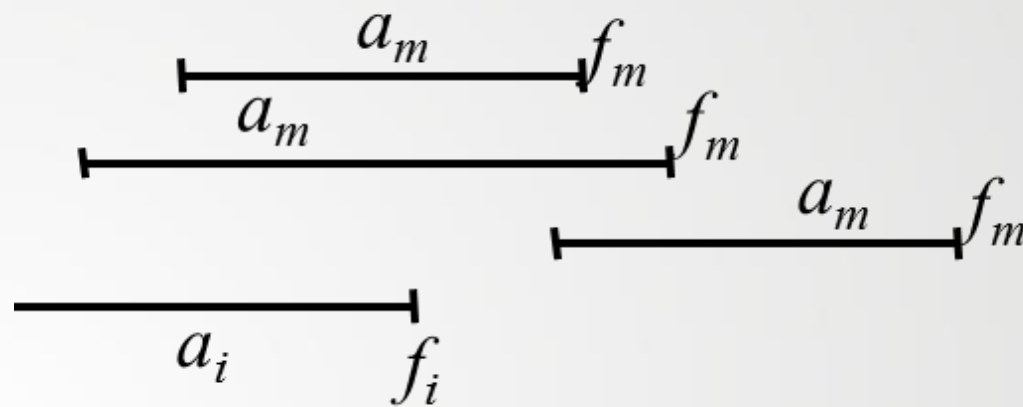
- 我们通常选择最先结束的任务
 - 贪心选择将剩余时间最大化
 - 各个任务的结束时间是严格递增的



一个递归算法

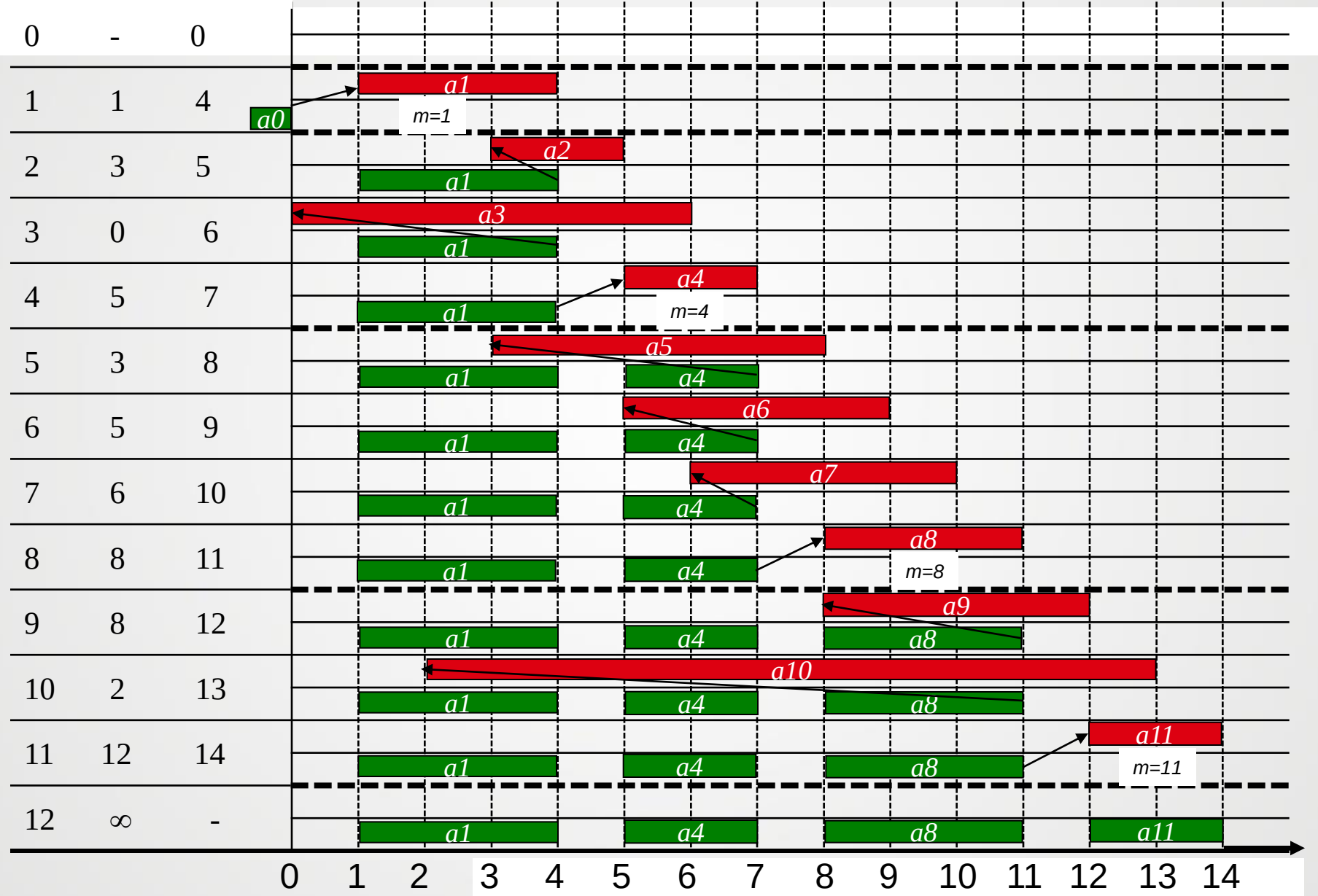
RecursiveTaskSelect(s, f, i, j)

- 1 $m \leftarrow i + 1$
- 2 **while** $m < j$ and $s_m < f_i$ **do**
- 3 $m \leftarrow m + 1$
- 4 **if** $m < j$ **then return** $\{a_m\} \cup \text{RecursiveTaskSelect}(s, f, m, j)$
- 5 **else return** $\emptyset$



- 任务按照其结束时间升序排序
- 运行时间:
 - $\Theta(n)$ – 每一个任务只检查一次
- **初始调用:** RecursiveTaskSelect($s, f, 0, n+1$)

例子





一个迭代算法

GreedyTaskSelect(s, f)

1 $A \leftarrow \{a_1\}$

2 $i \leftarrow 1$

3 **for** $m \leftarrow 2$ **to** n **do**

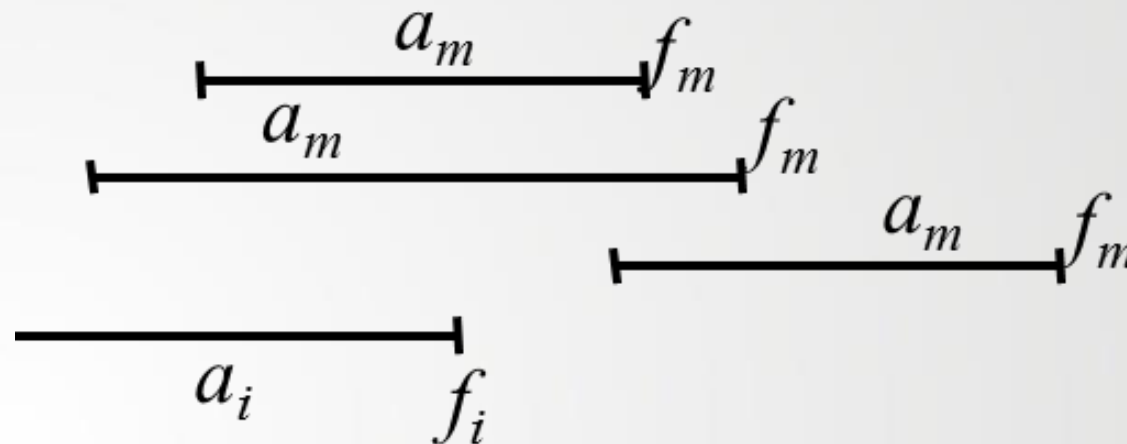
4 **if** **then**

5 $A \leftarrow A \cup \{a_m\}$

6 $i \leftarrow m$

7 **return** A

- 假设任务都按照其结束时间升序排序
- 运行时间: $\Theta(n)$ – 每一个任务只检查一次





贪心算法的基本步骤

1. 确定问题的最优子结构性质。
2. 定义递归解。
3. 证明递归的任一步，最优选择就是贪心选择。
4. 证明做了贪心选择后，只留下一个非空子问题。
5. 设计一个实现贪心策略的递归算法。
6. 把递归算法转化为迭代算法。



4.2 贪心算法的基本要素

基本思想：

- 从问题的某一个初始解出发，通过一系列的贪心选择——当前状态下的局部最优选择，逐步逼近给定的目标，尽可能快地求得更好的解。
- 在贪心算法 (greedy method) 中采用逐步构造最优解的方法。在每个阶段，都作出一个按某个评价函数最优的决策，该评价函数最优称为 贪心准则 (greedy criterion)。
- 贪心算法的正确性，就是要证明按贪心准则求得的解是全局最优解。



4.2 贪心算法的基本要素

本节着重讨论可以用贪心算法求解的问题的一般特征。

对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢？这个问题很难给予肯定的回答。

但是，从许多可以用贪心算法求解的问题中看到这类问题一般具有2个重要的性质：**贪心选择性质**和**最优子结构性质**。



4.2 贪心算法的基本要素

1、贪心选择性质

所谓贪心选择性质是指所求问题的**整体最优解**可以通过一系列**局部最优**的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。

动态规划算法通常以**自底向上**的方式解各子问题，而贪心算法则通常以**自顶向下**的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。

对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。



4.2 贪心算法的基本要素

2、最优子结构性质

当一个问题的最优解包含其子问题的最优解时，称此问题具有**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。



贪心算法的正确性

■ 贪心选择

- 一个局部的贪心选择可以导致全局的最优解

■ 最优子结构

- 我们知道通过作出贪心选择来获得一个子问题

贪心选择 + 子问题最优解 = 整个问题的最优解



任务的选择

- 贪心选择 a_m

- 存在一个最优解包含贪心选择: a_m 在 S_{ij} 中有最早的完成时间

- 最优子结构:

如果 S_{ij} 的最优解包含 $a_m \Rightarrow$ 它必须包含 S_{ik} 和 S_{kj} 的最优解,
类似的, $a_m + S_{im}$ 的最优解 $\Rightarrow$ 总的最优解.



动态规划算法与贪心算法

■ 动态规划

- 我们每一步都要做选择
- 选择取决于子问题的解
- 自底向上，从小到大

■ 相同点：最优子结构

■ 贪心算法

- 先作出一个贪心选择
- 解决由选择导出的子问题
- 我们做的选择依赖于前面的选择，而不是子问题的解
- 自顶向下，问题逐步减少



4.2 贪心算法的基本要素

3、贪心算法与动态规划算法的差异

贪心算法和动态规划算法都要求问题具有最优子结构性，这是2类算法的一个共同点。但是，对于具有**最优子结构**的问题应该选用贪心算法还是动态规划算法求解？是否能用动态规划算法求解的问题也能用贪心算法求解？下面研究2个经典的**组合优化问题**，并以此说明贪心算法与动态规划算法的主要差别。

谢谢大家